



程序设计实践

The Practice of Programming

设计与实现



算法和数据结构

查找

排序

可增长
数组

链表

树

散列表



数组下标访问

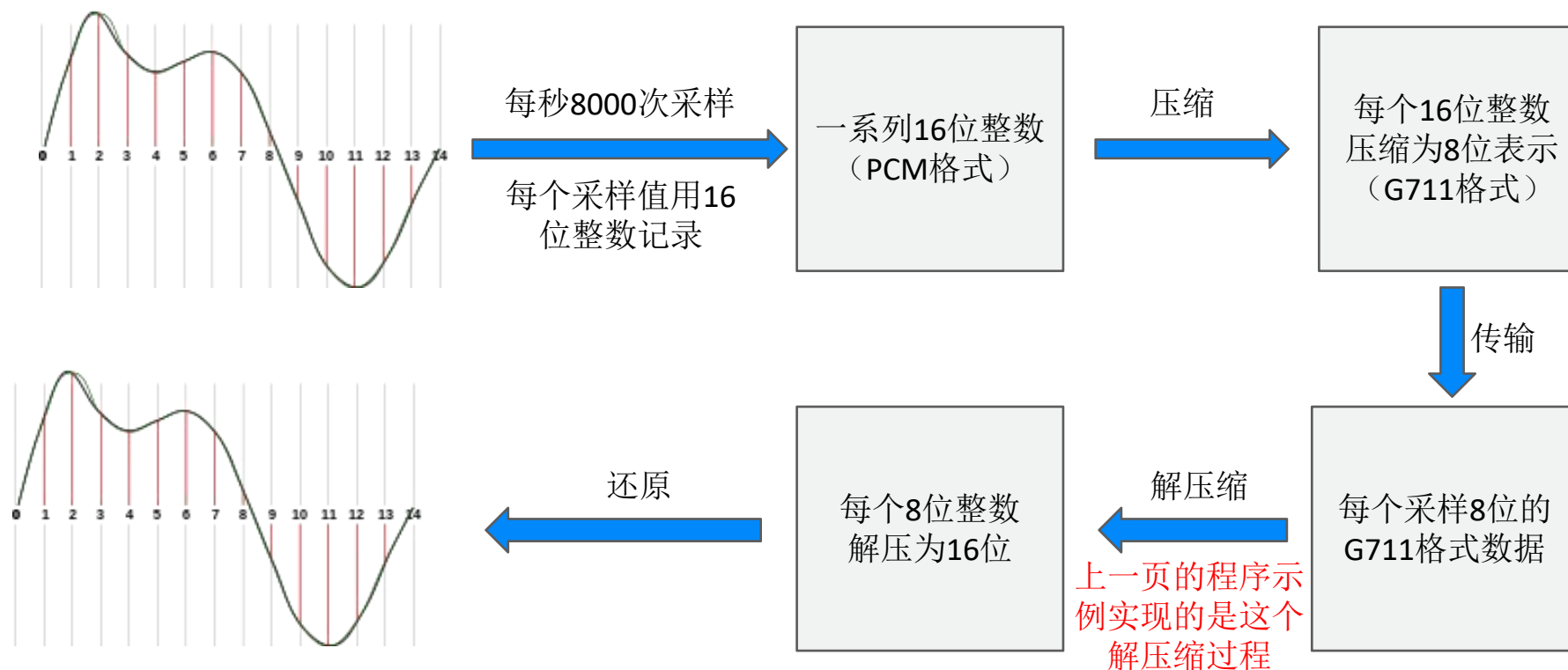
最简单，最快，最常用的“算法”

应用条件：

1. 要查询的数据的索引都是不太大的整数。
2. 这些整数稀疏程度不高。



语音编解码背景知识



因为G711编码后的取值，只有256种可能性，所以只要事先构造出一个映射表，就可以将复杂的压缩算法：

$$F^{-1}(y) = \text{sgn}(y) \begin{cases} \frac{|y|(1+\ln(A))}{A}, & |y| < \frac{1}{1+\ln(A)} \\ \frac{\exp(|y|(1+\ln(A))-1)}{A}, & \frac{1}{1+\ln(A)} \leq |y| \leq 1. \end{cases}$$

变为一个简单的数组下标访问



数组下标访问（续）

// G711语音解码程序:

```
static short _alawDecompressTable[256] =
{
    0xEA80, 0xEB80, 0xE880, 0xE980, 0xEE80, 0xEF80, 0xEC80, 0xED80,
    0xE280, 0xE380, 0xE080, 0xE180, 0xE680, 0xE780, 0xE480, 0xE580,
    0xF540, 0xF5C0, 0xF440, 0xF4C0, 0xF740, 0xF7C0, 0xF640, 0xF6C0,
    0xF140, 0xF1C0, 0xF040, 0xF0C0, 0xF340, 0xF3C0, 0xF240, 0xF2C0,
    0x03B0, 0x0390, 0x03F0, 0x03D0, 0x0330, 0x0310, 0x0370, 0x0350,
    ...
};

short alawToPcm(unsigned char sample)
{
    return _alawDecompressTable[sample];
}
```




使用了256个整数构成的数组，存放了上一
页表格中的数据，每一位0或者1，对
应字符的isxxx函数的bool返回值

字符ASC码
作为下标
访问数组

0x00
0x09(制表符)
...

... 0 0 0 0 0 0 0 0 0

...

... 1 0 1 1 0 0 0 0 0

...

0x31(1)

... 0 1 1 0 1 1 0 0 0

0x31(2)

... 0 1 1 0 1 1 0 0 0

0x32(2)

... 0 1 1 0 1 1 0 0 0

...

...

0x41(A)

... 0 1 1 0 1 0 1 0 1

0x42(B)

... 0 1 1 0 1 0 1 0 1

0x43(C)

... 0 1 1 0 1 0 1 0 1

...

...

0x58(X)

... 0 1 1 0 0 0 1 0 1

0x59(Y)

... 0 1 1 0 0 0 1 0 1

...

...

0x61(a)

... 0 1 1 0 1 0 1 1 0

0x61(B)

... 0 1 1 0 1 0 1 1 0

0x63(c)

... 0 1 1 0 1 0 1 1 0

...

...

例如isdigit('1'), 就找第49个整
数, 是32位的整数, 看看某一位
是否为1, 为1的话isdigit就返回
true, 否则返回false

isdigit('1') -> true

islower('A') -> false

islower('a') -> true

islower用的位

isprint用的位



顺序查找(Sequential Search) 简单，低效的查找算法

可能的应用场景：

1. 要查询的数据特别少，例如少于10条。
2. 被查找数据的排列顺序不能自由设定，例如字符串中查找。
3. 查询条件太复杂，以至于匹配条件的开销大于查找。
4. 更新频繁，却很少查找。
5. 时间复杂度 $O(n)$

有关字符串顺序查找的标准库：

1. C: strchr, strstr
2. C++: std::find
3. Java: String.indexOf



二分查找(Binary Search) 高效，但是对应用条件要求比较苛刻

应用条件：

1. 数据已经有序
2. 数据基本有序，查找前可以很快调整成有序
3. 数据无序，但是排序后更新次数远小于查找次数。
4. 时间复杂度 $O(\log_2 n)$

二分查找标准库：

1. C++: `std::binary_search`
2. Java: `Arrays.binarySearch`



排序(Sorting)

排序算法:

1. 选择、冒泡、插入排序
2. 快速排序
3. 其它排序算法...

库:

1. C: `qsort` (快速排序)
2. C++: `std::sort`
3. Java: **`Array.sort`**
3. Java: **`Array.sort`**

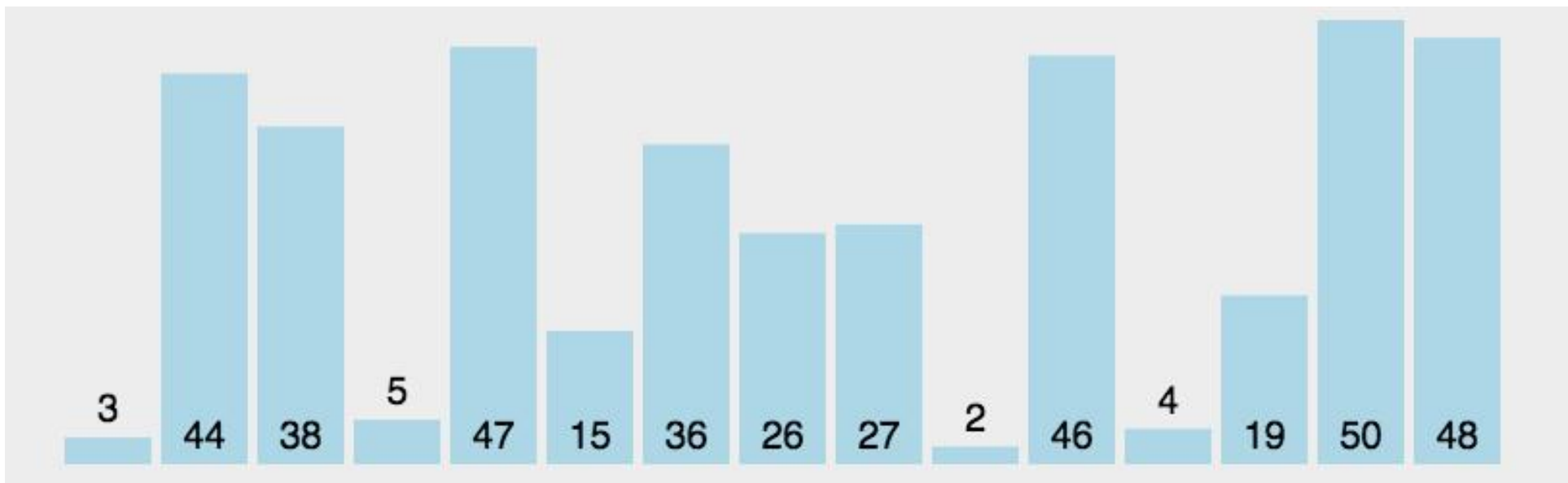
常见排序算法及动画演示

<https://www.cnblogs.com/onepixel/articles/7674659.html>



排序(Sorting)

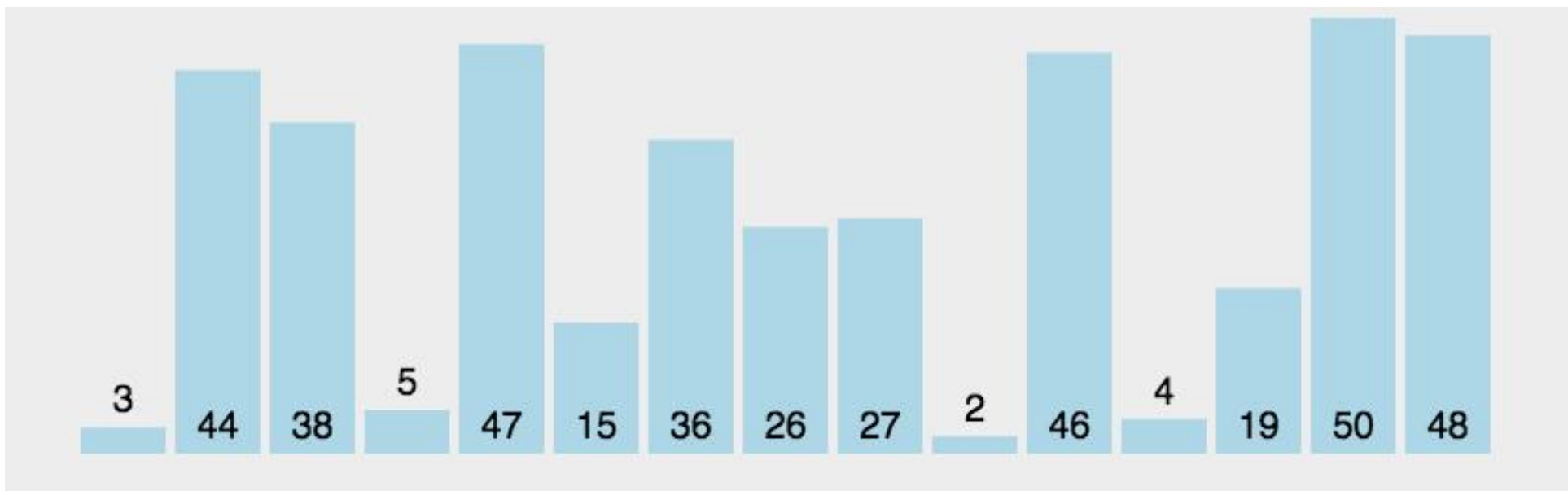
选择排序





排序(Sorting)

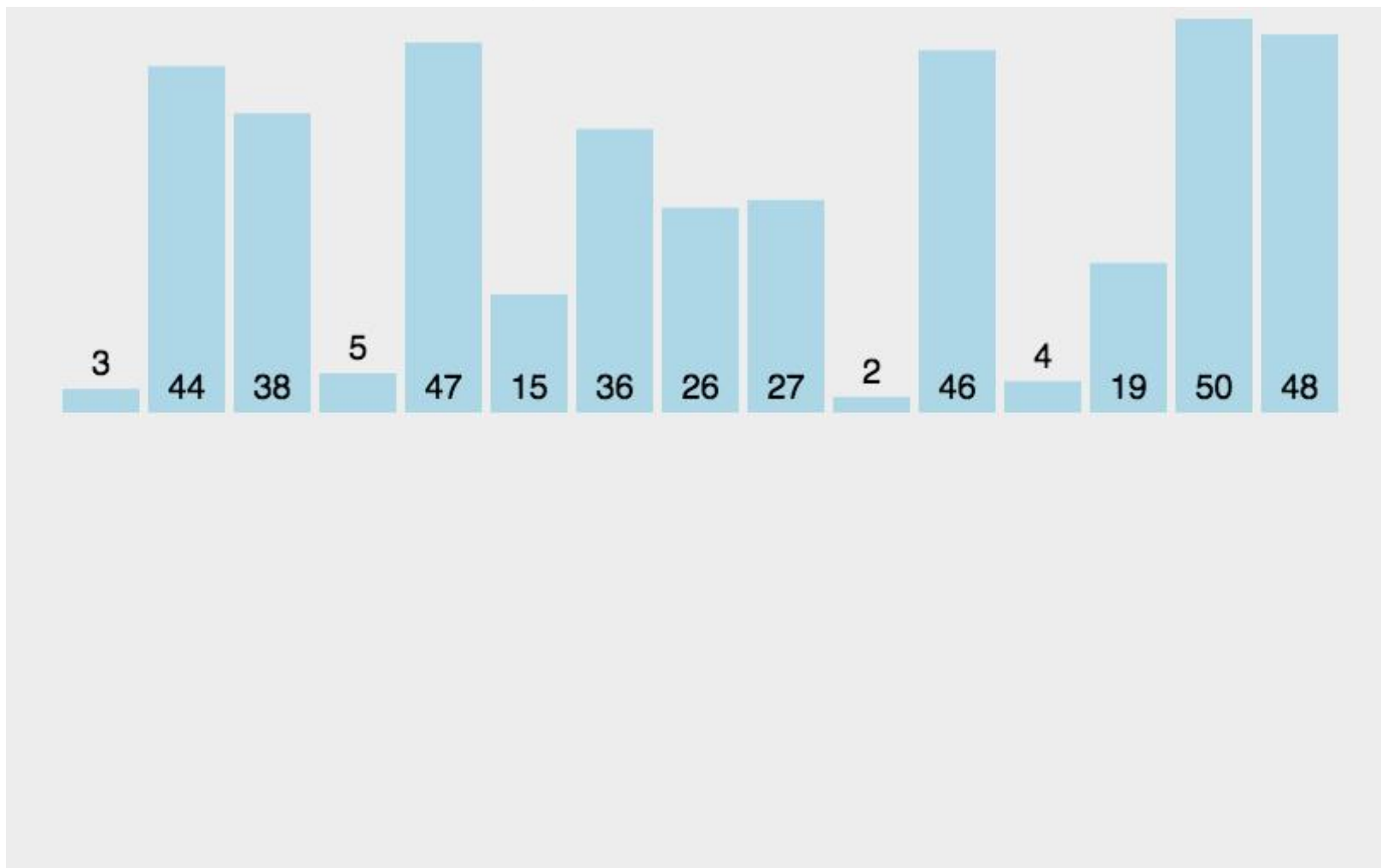
冒泡排序





排序(Sorting)

插入排序





快速排序(Quicksort) 诞生于1960年代的最好的排序算法

```
void quick_sort(int v[], int n){  
    if(n <= 1)  
        return;  
    swap(v, 0, rand() % n);  
    int last = 0;  
  
    for(int i = 1; i < n; i++) {  
        if(v[i] < v[0])  
            swap(v, ++last, i);  
    }  
    swap(v, 0, last);  
  
    quick_sort(v, last);  
    quick_sort(v + last + 1, n - last - 1);  
}
```

```
void swap(int v[], int i, int j){  
    int temp = v[i];  
    v[i] = v[j];  
    v[j] = temp;  
}
```

如果元素个数小于等于1，则不用排了

随机取数组中间一个元素，作为支点元素，
暂且和第0个元素交换

last以及之前的元素，都确定比支点元素
要小，last暂且初值为0

从1到最后扫描一遍，找出所有比支点元
素小的，排到++last那里去

把此前暂且交换到开头的支点元素与last
交换，至此支点元素之前的元素，都比它
小，之后的都比它大(或者等于)。以支点
元素为界，数组被拆分成两部分

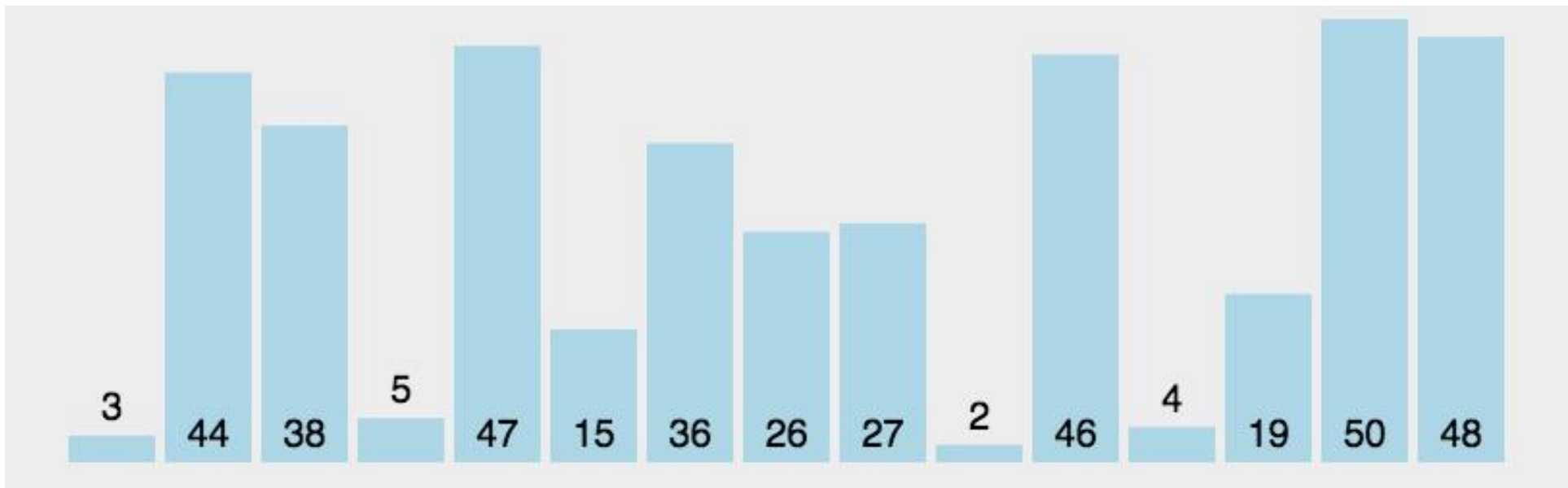
递归调用，对前半部分排序

递归调用，对后半部分排序



排序(Sorting)

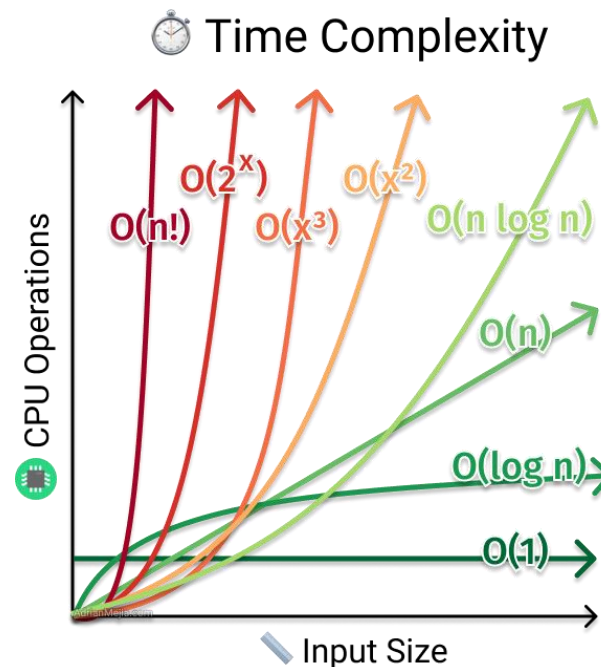
快速排序



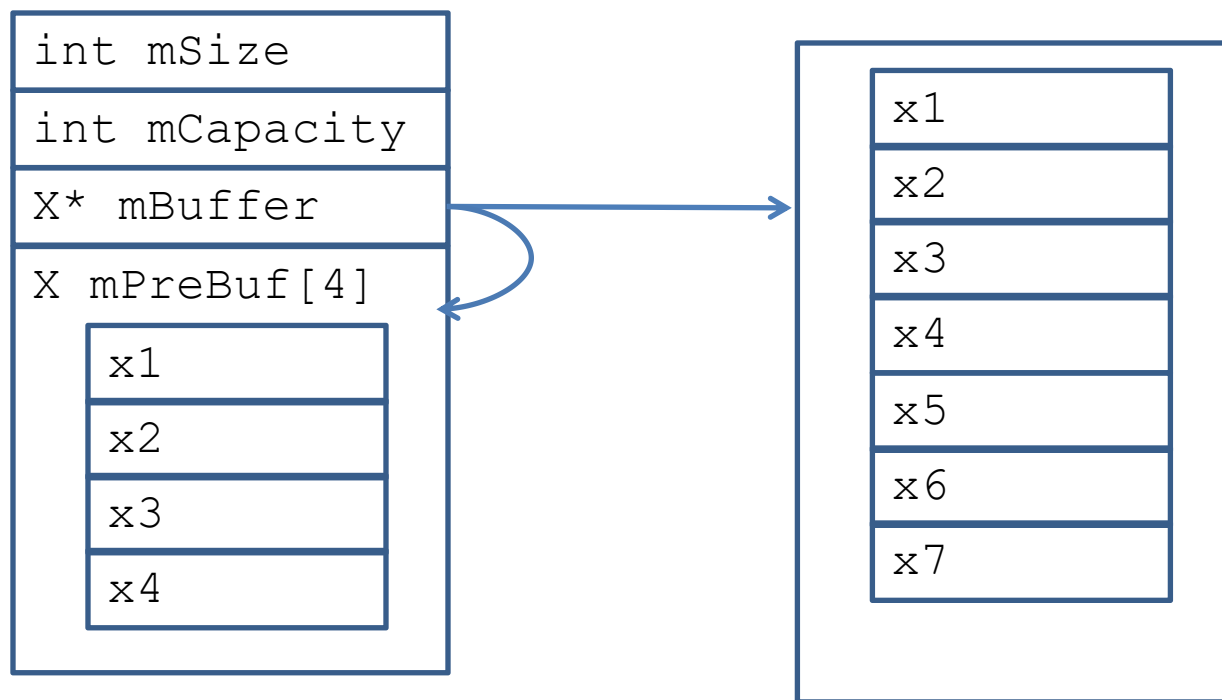
- 棕色: 已确定排序位置的元素
- 黄色: 当前正在处理的子序列的支点元素
- 红色: 正在处理的元素
- 蓝色: 子序列中尚未与支点元素比较的元素
- 绿色: 子序列中已与支点比过, 并且值小于支点的元素
- 紫色: 子序列中已与支点比过, 并且值大于支点的元素

算法复杂度大O记法

记法	名称	举例
$O(1)$	常数(constant)	数组下标访问
$O(\log n)$	对数(logarithmic)	二分查找
$O(n)$	线性(linear)	顺序查找
$O(n \log n)$	$n \log n$	快速排序
$O(n^2)$	平方(quadratic)	简单排序
$O(n^3)$	立方(cubic)	矩阵乘法
$O(2^n)$	指数(exponential)	集合划分



可增长数组(Growing Array)

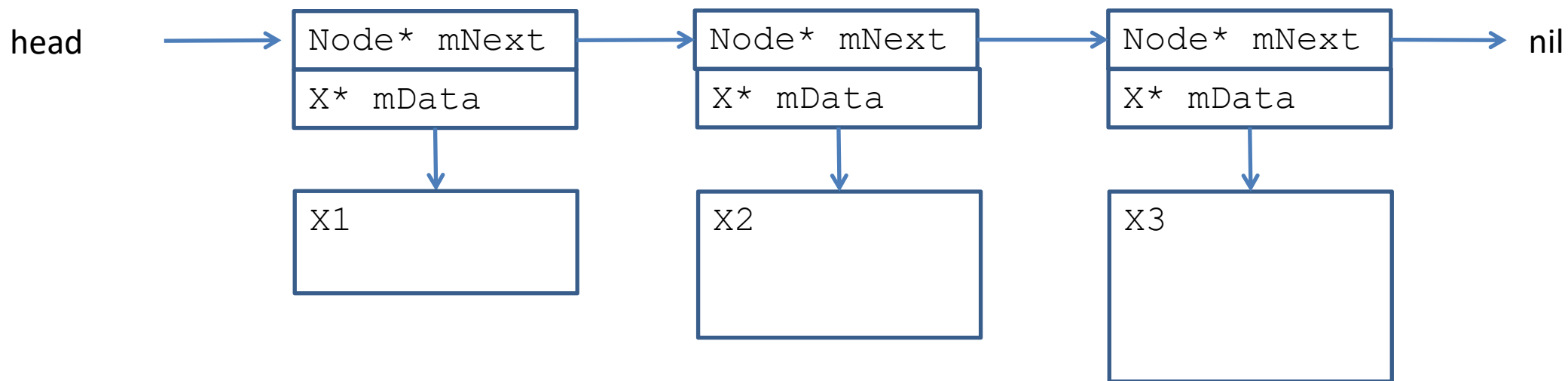
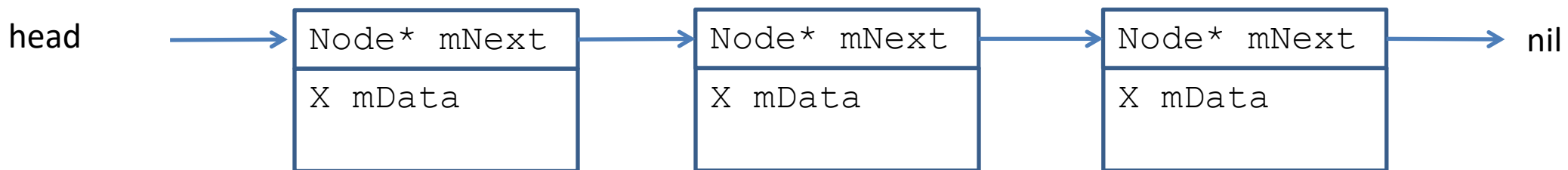


库:

1. C++: `std::string`
2. Java: `String`



链表(List)

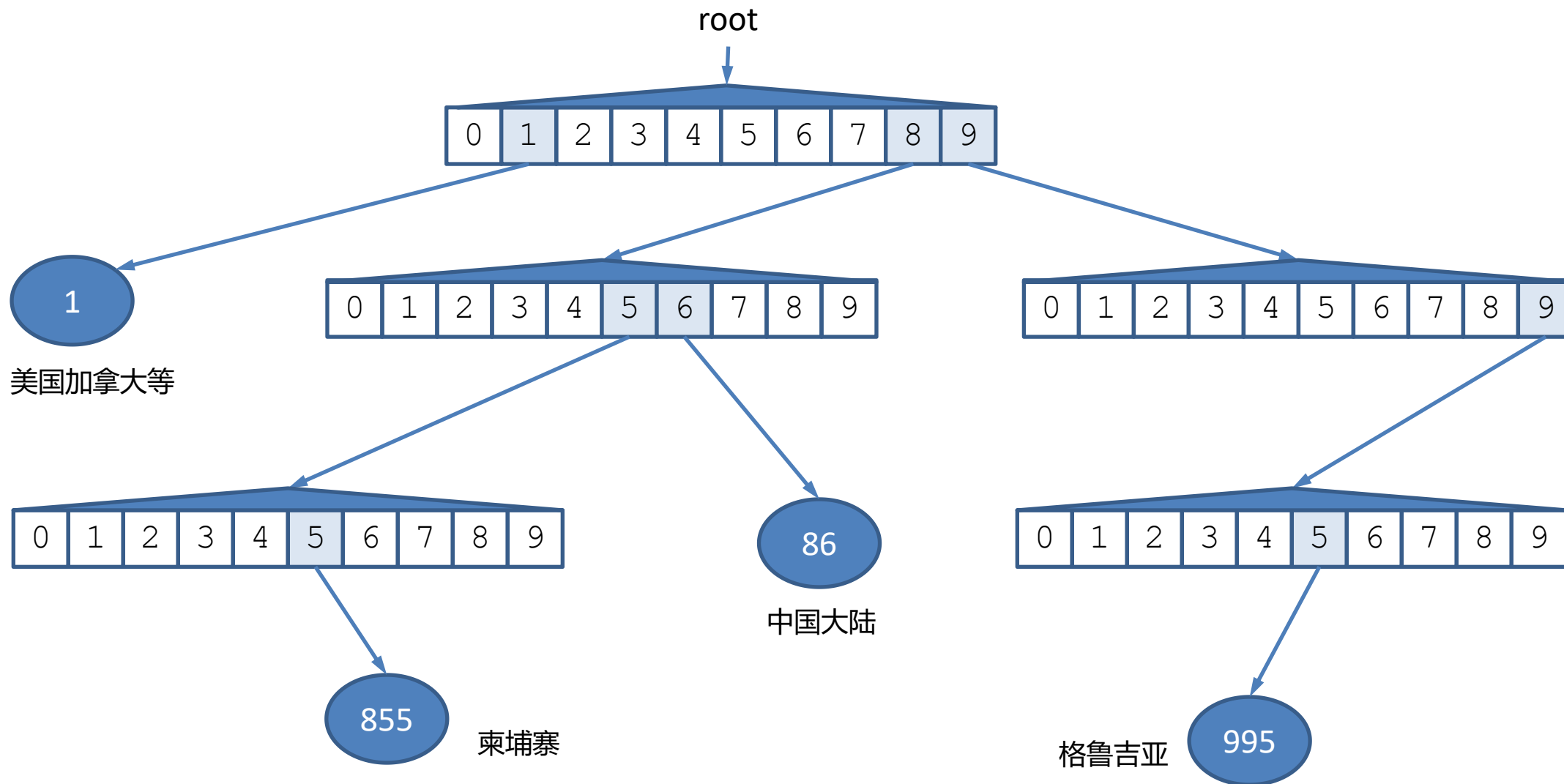


C++标准库: `std::list`



树

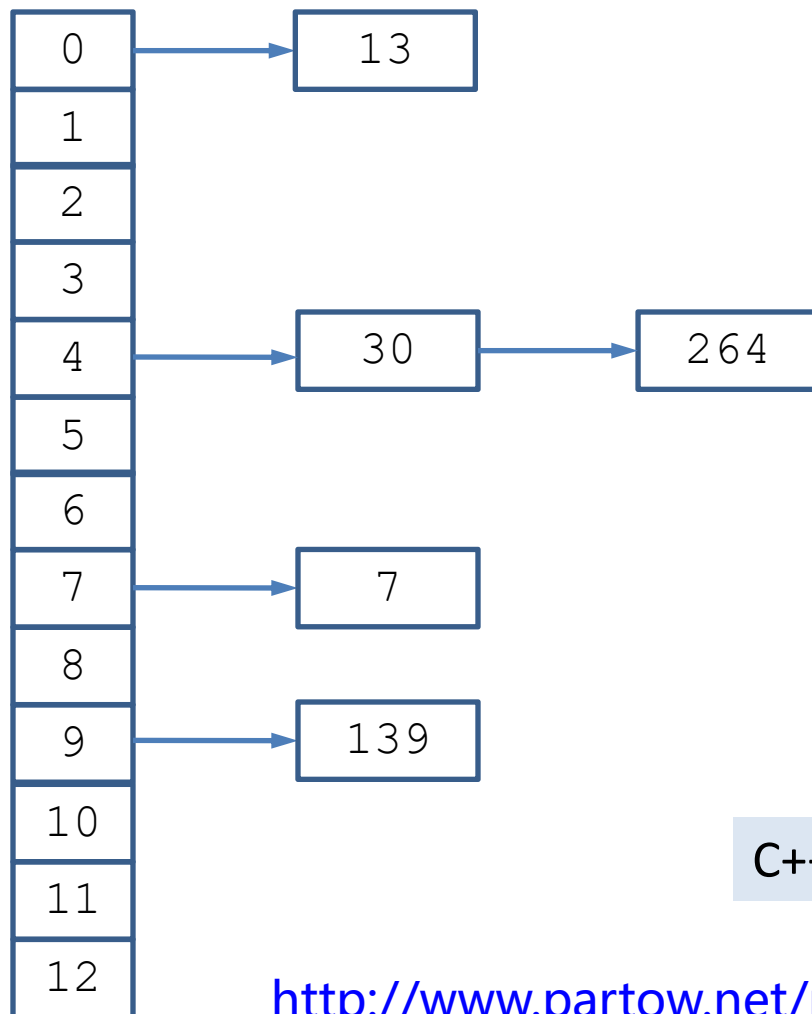
问题：如何从一个带地区号的电话号码中求出地区号？
(例如12024952266, 861062282297, 9952259000)





散列表(Hash Table)

计算机科学领域伟大的发明之一
应用最广泛的一个数据结构



索引：整数
散列数组大小：13
散列算法：(n % 13)

索引：字符串
散列数组大小：?
散列算法：?

C++标准库：std::hash_map

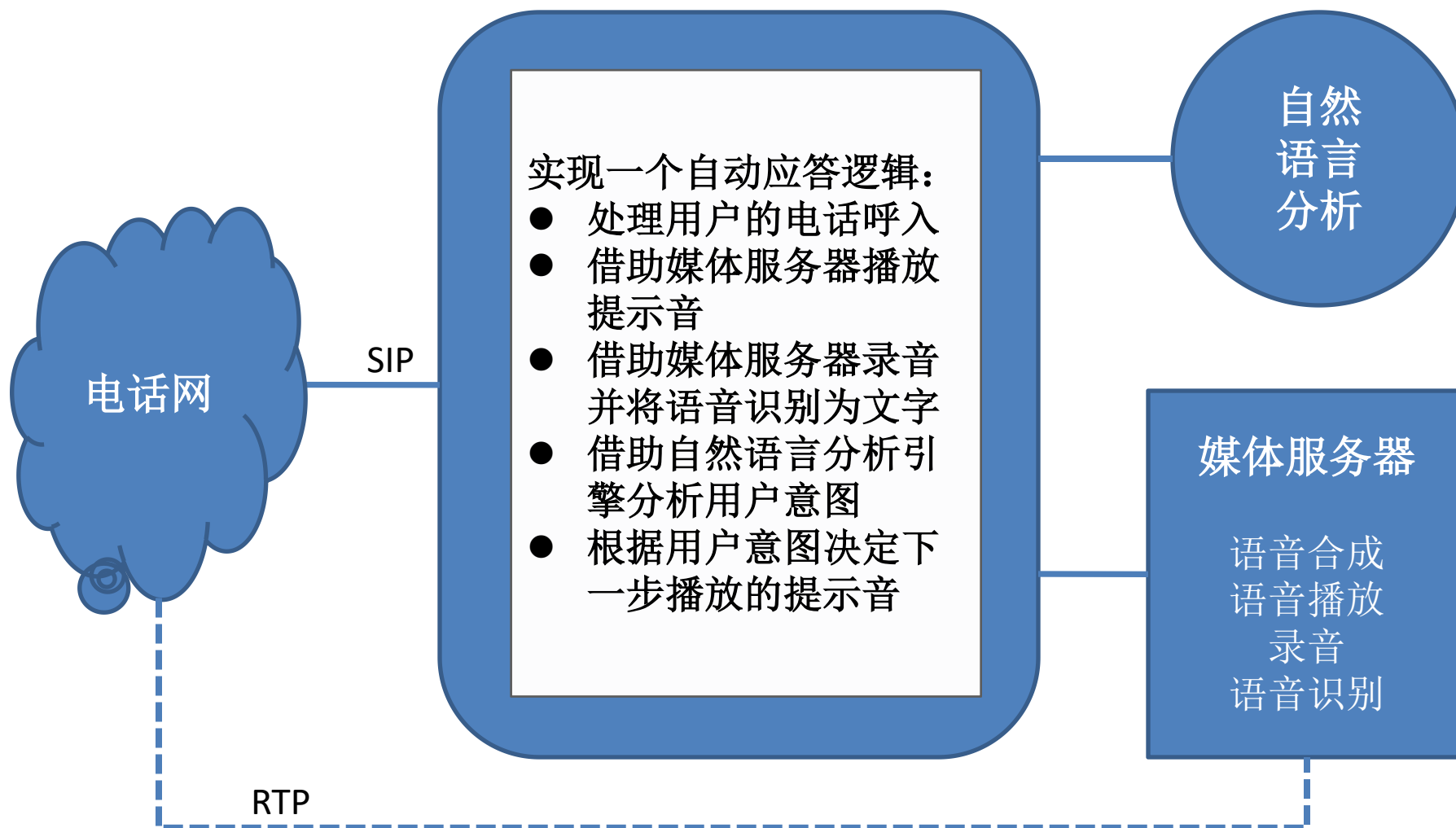
<http://www.partow.net/programming/hashfunctions/>
这个网站有很多散列算法的例子



设计实现一个脚本解释器

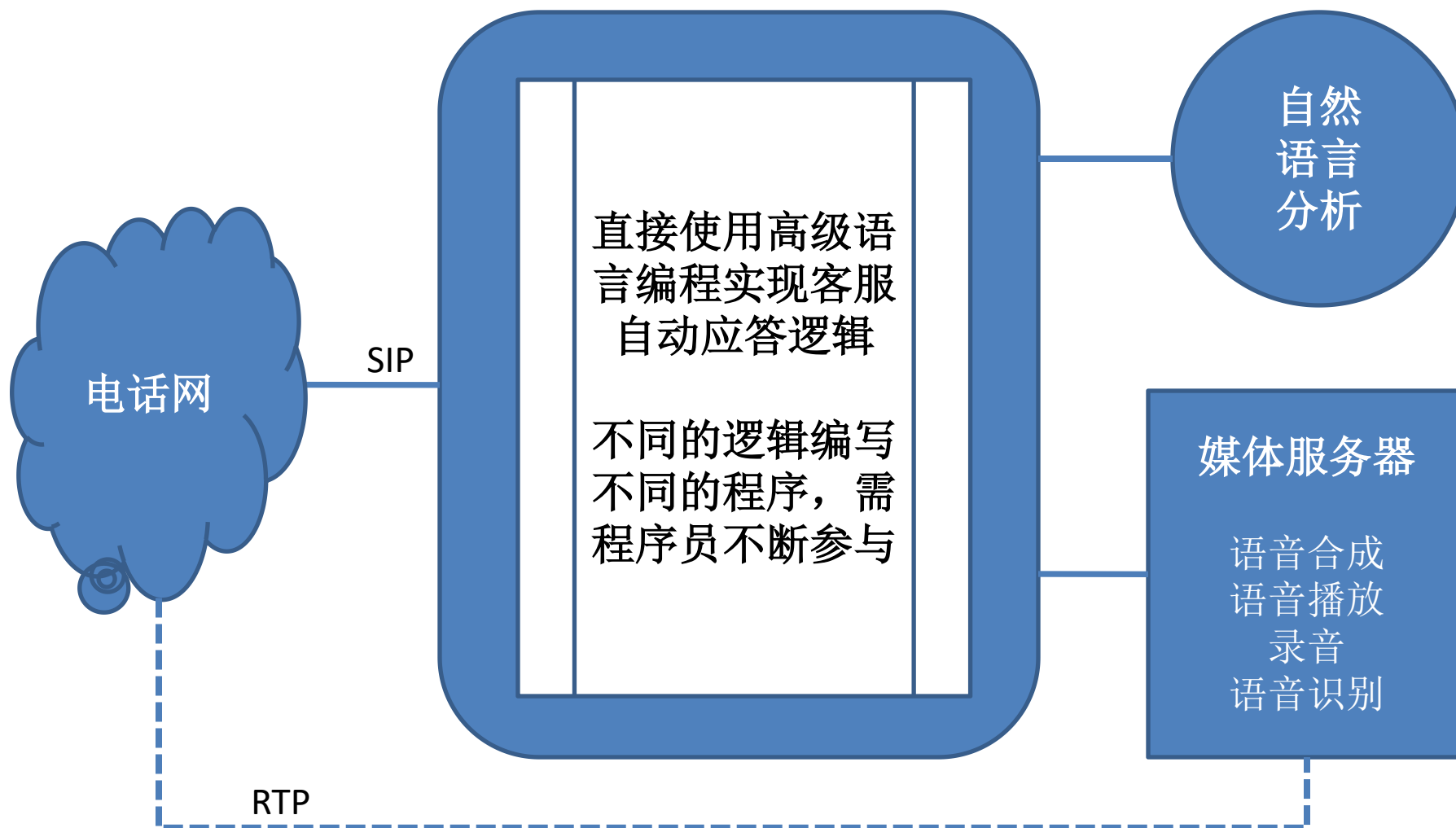
需求

实现一个语音客服机器人



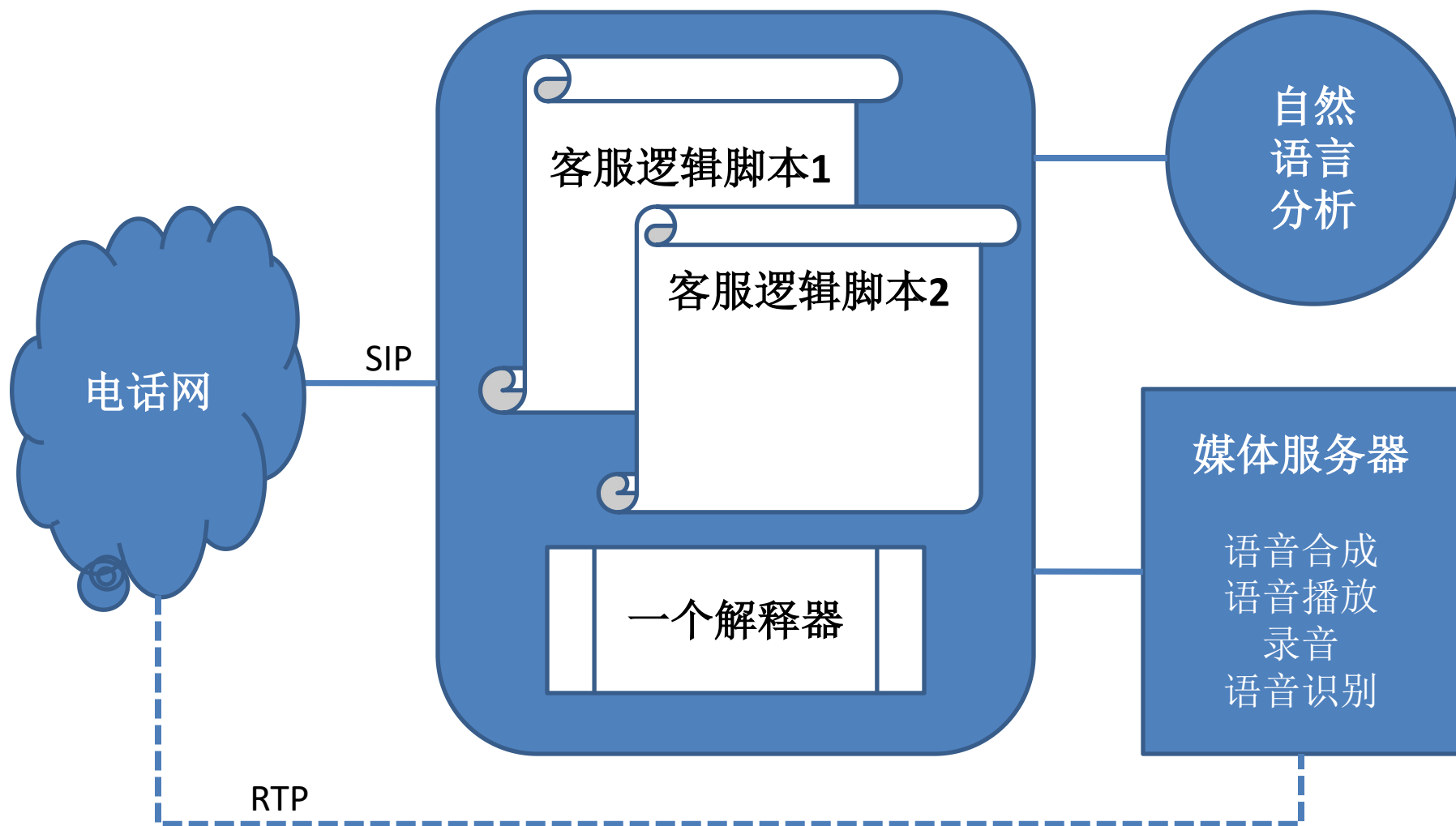


方案1

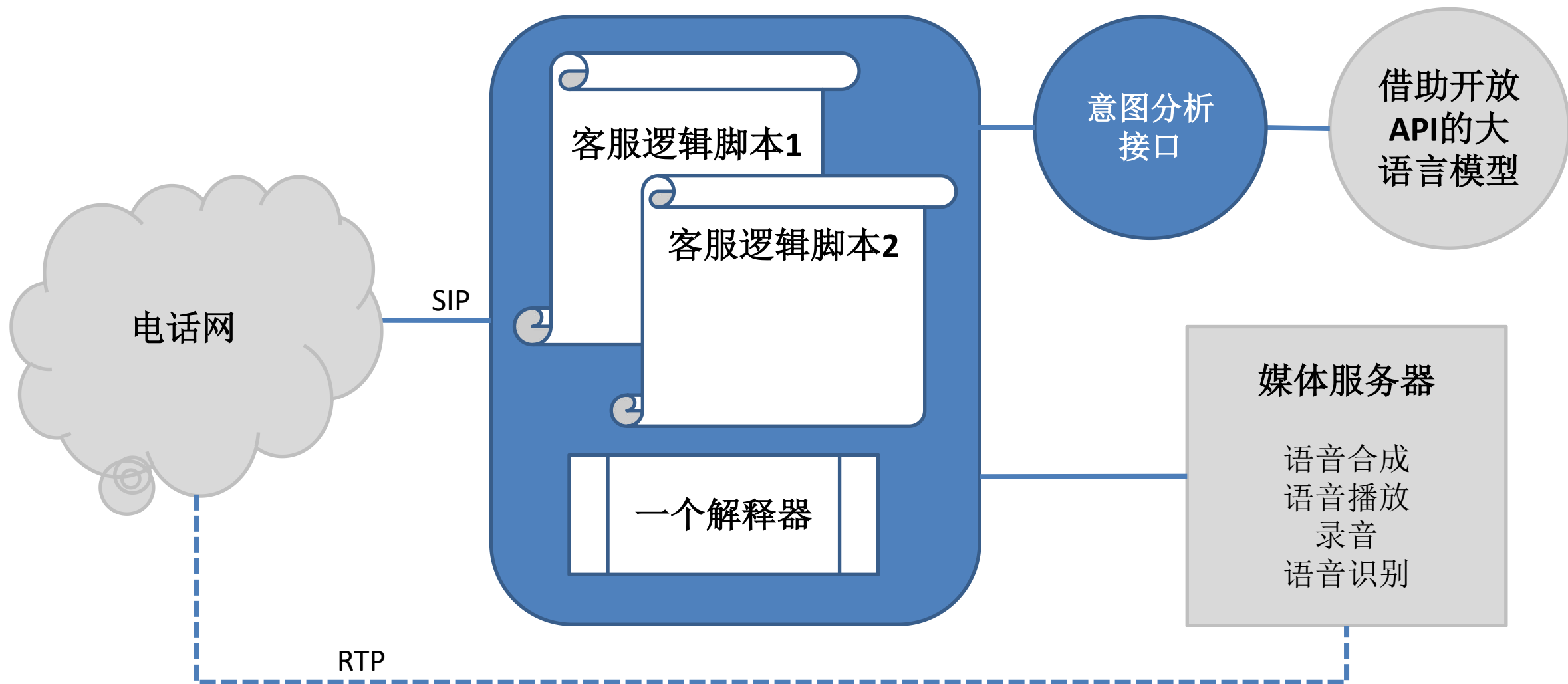




方案2



我们要做的部分



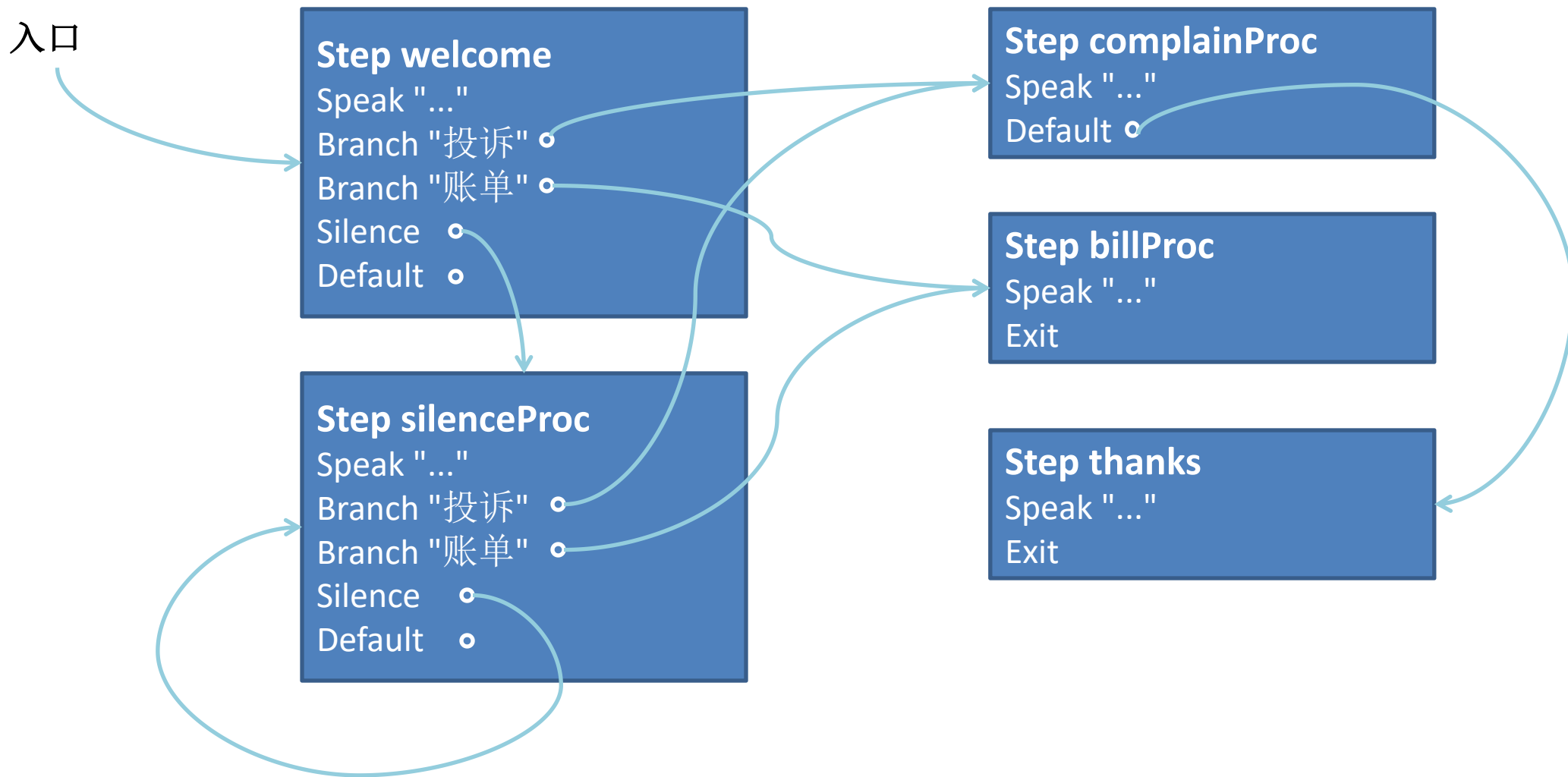


客服 逻辑 脚本 示例

```
Step welcome
  Speak $name + "您好, 请问有什么可以帮您?"
  Listen 5, 20
  Branch "投诉", complainProc
  Branch "账单", billProc
  Silence silenceProc
  Default defaultProc
Step complainProc
  Speak "您的意见是我们改进工作的动力, 请问您还有什么补充?"
  Listen 5, 50
  Default thanks
Step thanks
  Speak "感谢您的来电, 再见"
  Exit
Step billProc
  Speak "您的本月账单是" + $amount + "元, 感谢您的来电, 再见"
  Exit
Step silenceProc
  Speak "听不清, 请您大声一点可以吗"
  Branch "投诉", complainProc
  Branch "账单", billProc
  Silence silenceProc
  Default defaultProc
Step defaultProc
  ....
```



脚本的含义





脚本的语义动作

Step: 完整表示一个步骤的所有行为

Speak:

计算表达式合成一段文字

调用媒体服务器进行语音合成并播放

Listen:

调用媒体服务器对客户说的话录音，并进行语音识别

语音识别的结果调用“自然语言分析服务”分析客户的意愿

Branch:

对客户的意愿进行分支处理，不同的意愿，跳转到不同的Step

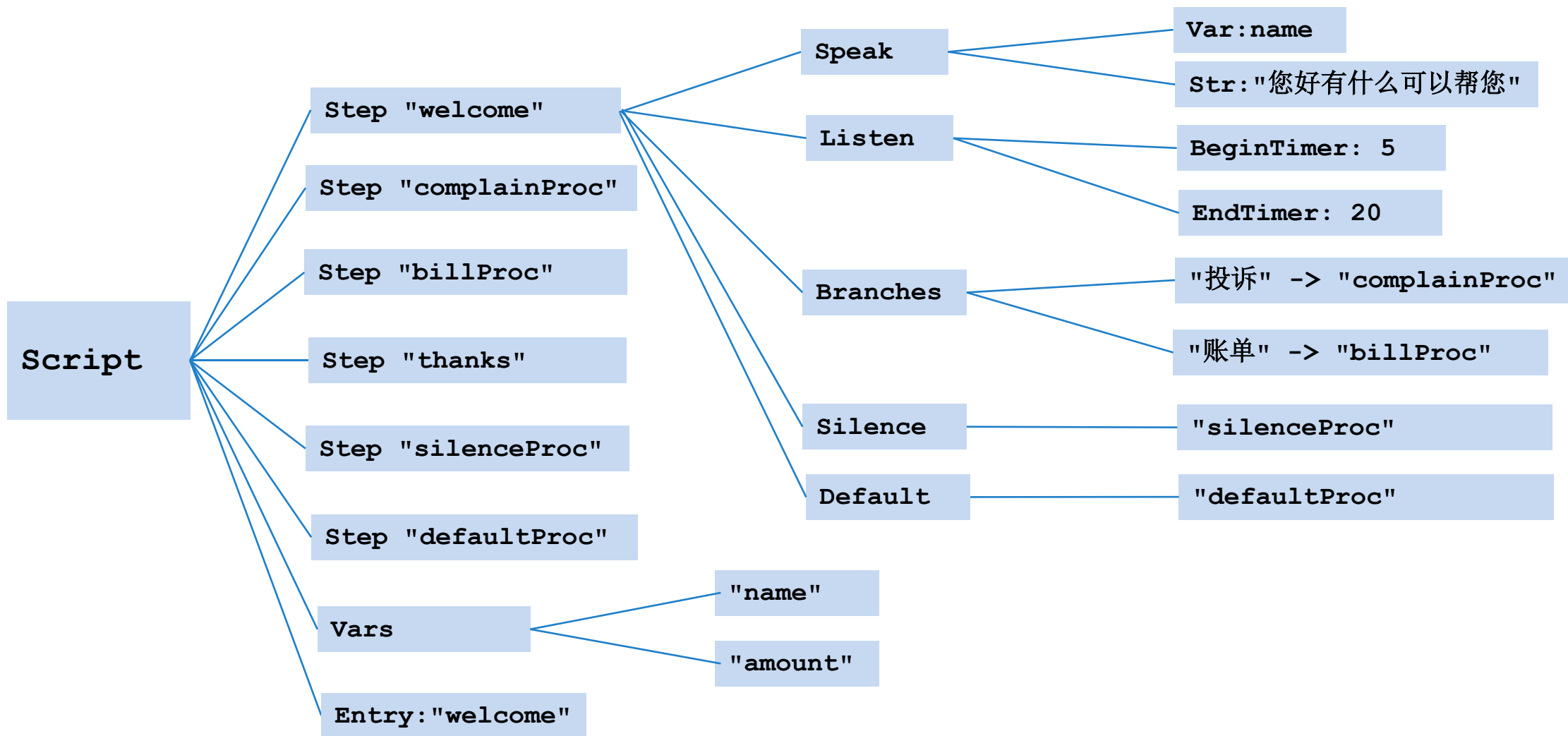
Silence: 如果用户不说话，应该跳转到哪个Step

Default: 如果客户意愿没有相应匹配，应该跳转到哪个Step

Exit: 结束对话

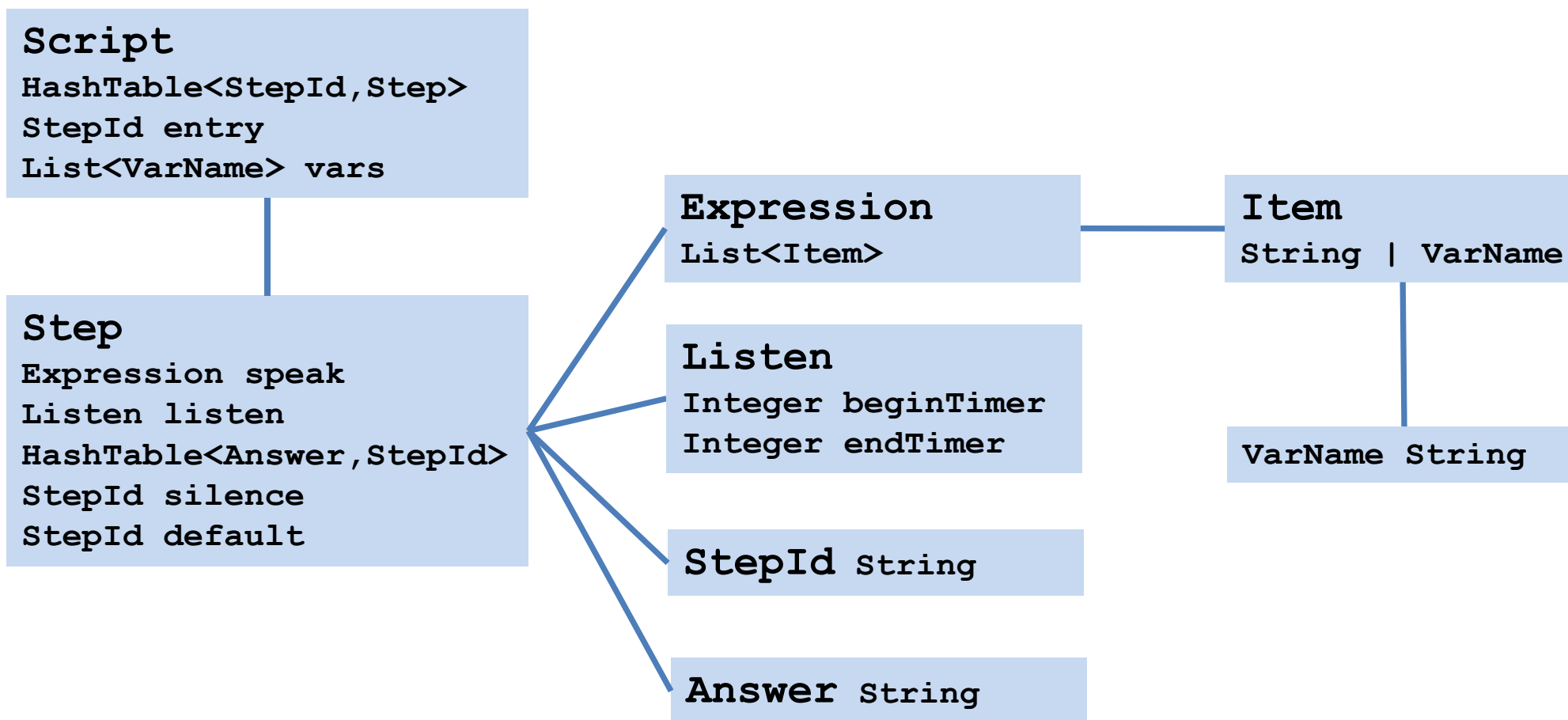


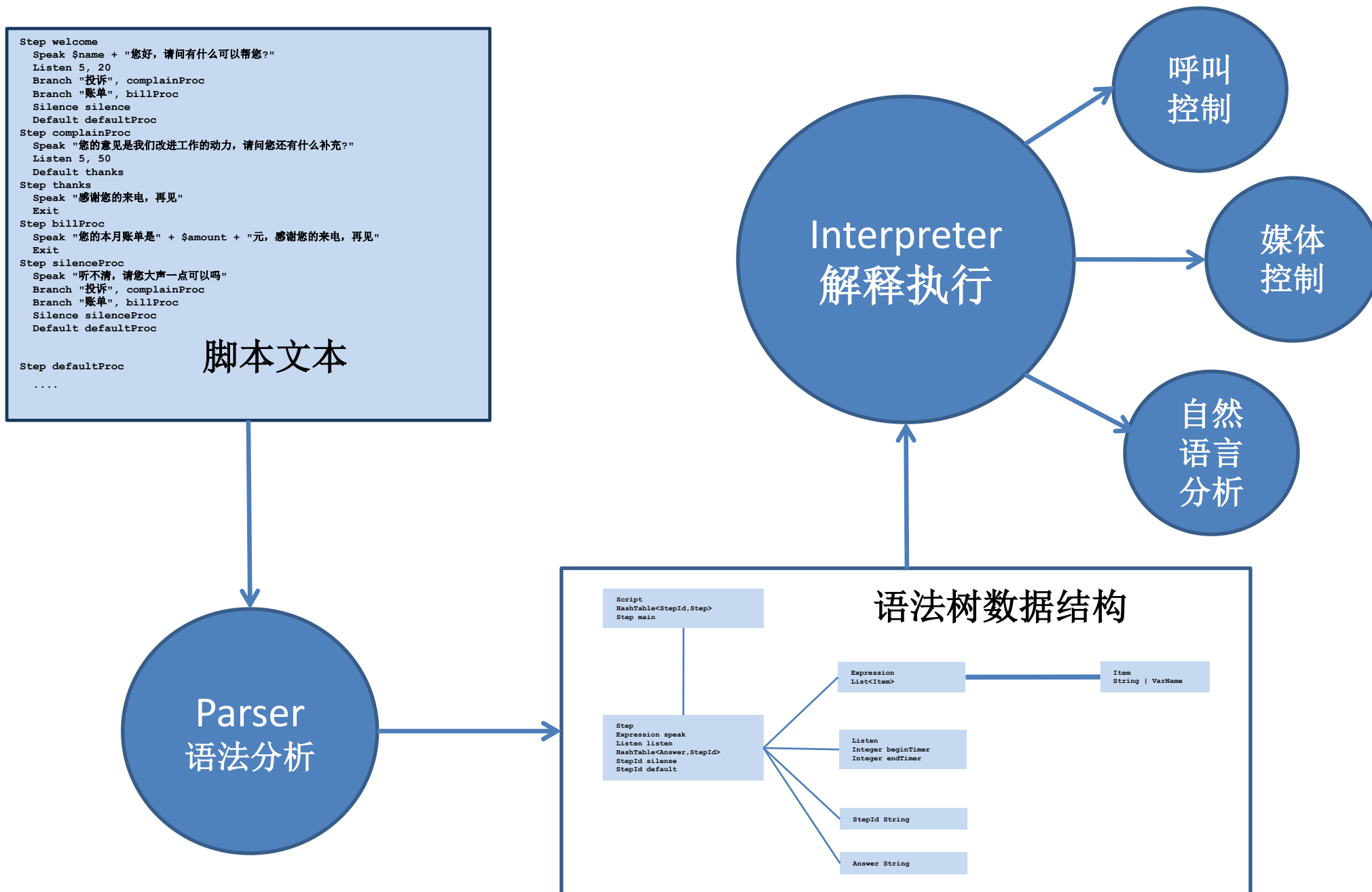
将脚本的语法元素抽象为树形结构





存储语法树的数据结构







```
Step welcome
Speak $name + "您好, 请问有什么可以帮您?"
Listen 5, 20
Branch "投诉", complainProc
Branch "账单", billProc
Silence silence
Default defaultProc
Step complainProc
Speak "您的意见是我们改进工作的动力, 请问您还有什么补充?"
Listen 5, 50
Default thanks
Step thanks
Speak "感谢您的来电, 再见"
Exit
Step billProc
Speak "您的本月账单是" + $amount + "元, 感谢您的来电, 再见"
Exit
Step silenceProc
Speak "听不清, 请您大声一点可以吗"
Branch "投诉", complainProc
Branch "账单", billProc
Silence silenceProc
Default defaultProc

Step defaultProc
....
```

脚本文本

Parser
语法分析

Interpreter
解释执行

这里简化一下

显示Speak内容

输入客户意愿

语法树数据结构

Script
HashTable<StepId, Step>
Step main

Step
Expression speak
Listen listen
HashTable<Answer, StepId>
StepId silence
StepId default

Expression
List<Item>

Item
String | VarName

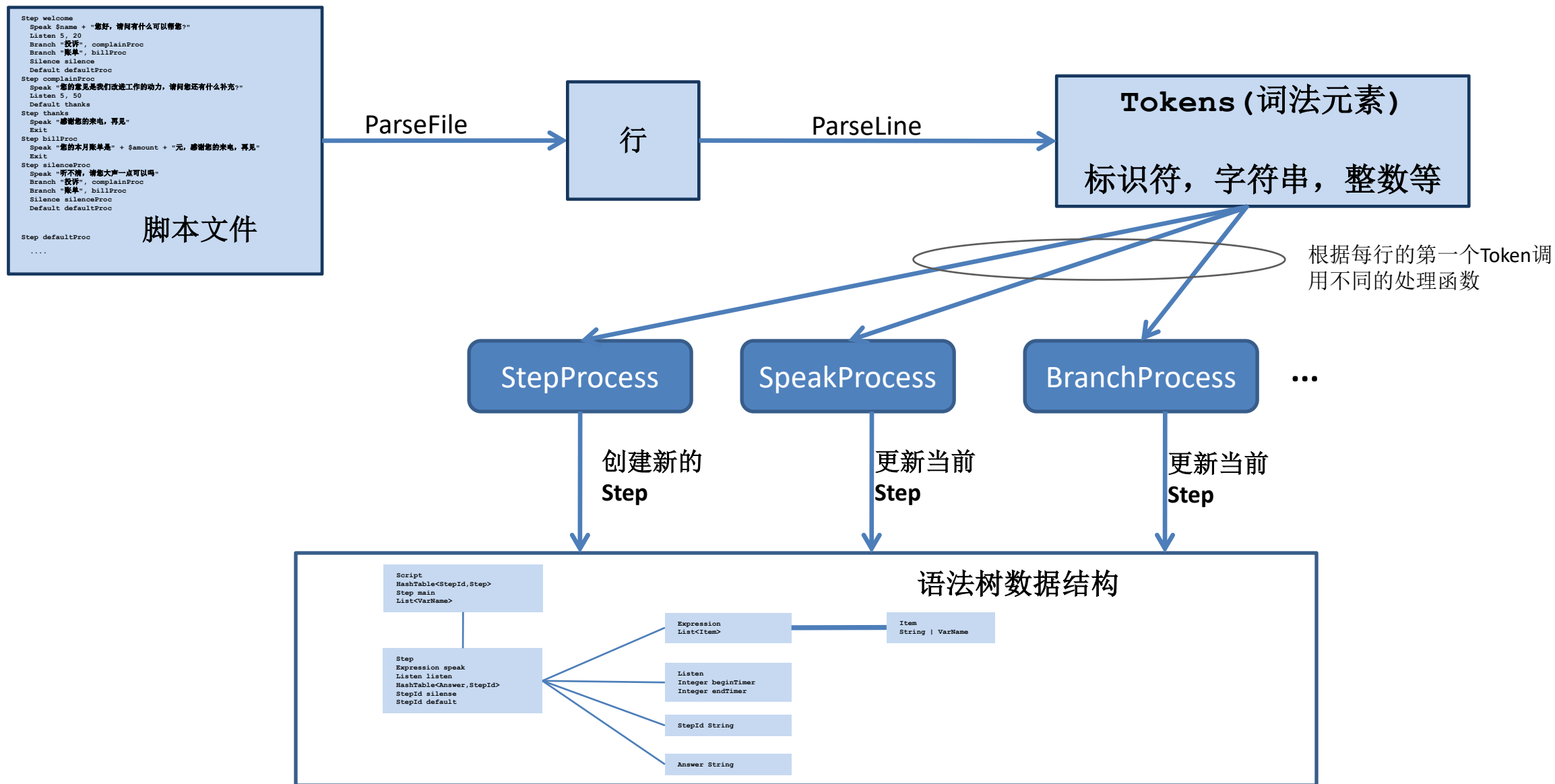
Listen
Integer beginTimer
Integer endTimer

StepId String

Answer String



Parser的实现





Parser的实现

ParseFile(fileName) :

打开文件

读取文件的每一行line:

line.trim() 删除行首空白

 忽略空行

 忽略'#'开头的注释行

ParseLine(line) 处理一行

关闭文件

ParseLine(line) :

读取一行中空白分割的每一个token:

 遇到'#'开头的token则处理结束（忽略行尾注释）

 获得标识符，字符串或者操作符几类token

 将token加入到List中

ProcessTokens(token[])



Parser的实现

ProcessTokens (token[]) :

对List中的每一个token进行处理

根据token[0]分情况处理:

Step: ProcessStep(token[1])

Speak: ProcessSpeak(token+1)

Listen: ProcessListen (token[1], token[2])

Branch: ProcessBranch(token[1], token[2])

Silence: ProcessSilence(token[1])

Default: ProcessDefault (token[1])

Exit: ProcessExit ()

如果不是上述token则报错

ProcessStep (stepId) :

Script创建一个新的Step, 标识为stepId, <StepId, Step>加入HashTable

设置当前Step为新创建的Step

如果这是第一个Step, 则设置当前Step为Script的entry



Parser的实现

ProcessSpeak (token[]) :

token[] 是一个表达式，每个token可能是字符串，变量或者 '+'
ProcessExpression (token[]) 得到 Expression
将 Speak 以及对应的表达式存入当前的 Step

ProcessExpression (token[]) :

这么简单的表达式...
忽略掉加号，其它token追加到 Expression 中的 List<Item>
将变量名存入 Script 的 List<VarName> 中

ProcessListen (startTimer, stopTimer) :

构造 Listen (startTimer, stopTimer) 存入当前 Step

ProcessBranch (answer, nextStepId) :

将 answer 和 nextStepId 插入当前 Step 的 HashTable



Parser的实现

ProcessSilence (nextStepId) :

将当前Step的silence变量设置成nextStepId中的值

ProcessDefault (nextStepId) :

将当前Step的default变量设置成nextStepId中的值

ProcessExit() :

将当前Step设置为终结Step



Interpreter的实现

执行环境:

- 变量表
- 当前Step
- ...

解释程序:

接通电话, 连接媒体服务器, 获取脚本语法树, 创建执行环境
当前Step置为entryStep

循环针对当前Step做:

执行Speak (语音合成, 语音播放)

如果本步骤是终结步骤, 则结束循环, 断开通话

执行Listen (录音, 语音识别, 自然语言理解)

获得下一个StepId:

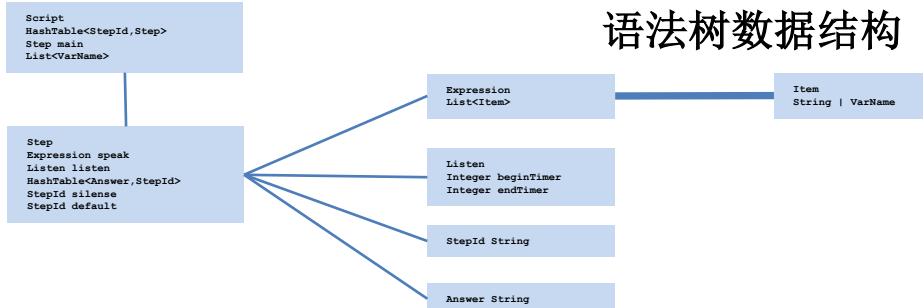
如果用户沉默, 则获得Silence的StepId

根据用户意向查找HashTable, 获得StepId

如果查不到则获得Default的StepId

将当前Step置为刚才获得的StepId对应的Step

语法树数据结构





Interpreter的实现（简化版）

执行环境：

- 变量表
- 当前Step
- ...

解释程序（简化版）：

获取脚本语法树，创建执行环境

当前Step置为entryStep

循环针对当前Step做：

执行Speak（输出到标准输出）

如果本步骤是终结步骤，则结束循环，断开通话

执行Listen（直接从标准输入读入用户意愿）

获得下一个StepId：

如果用户沉默，则获得Silence的StepId

根据用户意向查找HashTable，获得StepId

如果查不到则获得Default的StepId

将当前Step置为刚才获得的StepId对应的Step

语法树数据结构

```
Script
HashTable<StepId, Step>
Step main
List<VarName>
```

```
Step
Expression speak
Listen listen
HashTable<Answer, StepId>
StepId silence
StepId default
```

```
Expression
List<Item>
```

```
Item
String | VarName
```

```
Listen
Integer beginTimer
Integer endTimer
```

```
StepId String
```

```
Answer String
```



Interpreter的执行环境

当两个用户同时和机器人对话，解释器需要两个线程同时运行，每个线程服务一个用户。思考一下，哪些东西还是一份？那些东西需要两份？

脚本文件：同一份（两个用户执行的是同一个脚本）

脚本语法树：同一份（可以调用一次Parser形成，多个线程公用）

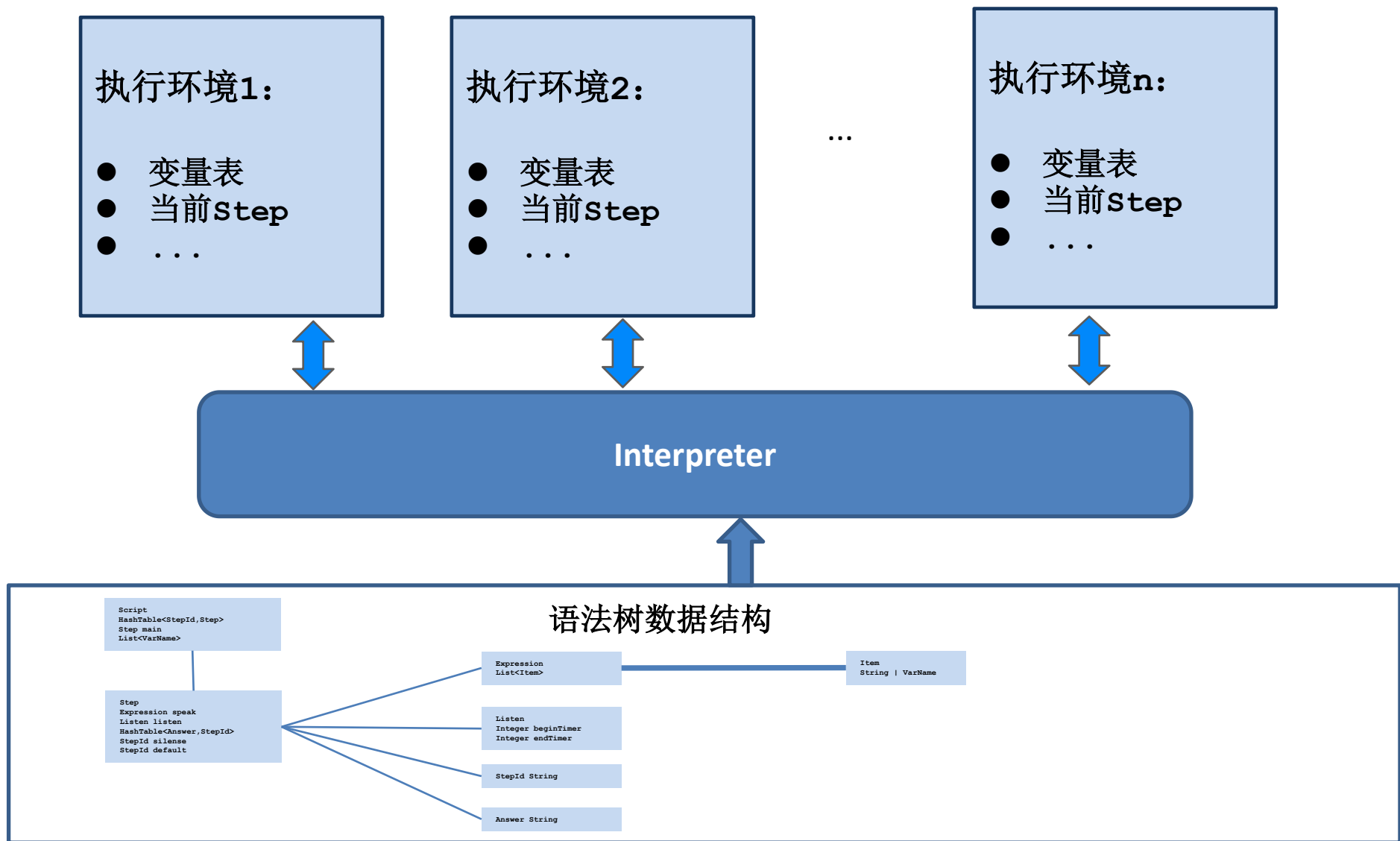
但是，不同的用户肯定有不同的姓名，不同的账单，所以表示姓名和账单的变量表，肯定是两份。

两个用户有先有后，有快有慢，脚本执行的当前状态（例如当前step）肯定是两份。

简单来说，上述这些有“两份”的数据，也就是Interpreter的每次执行都需要一个新的实例的数据，我们称之为Interpreter的执行环境，需要单独的数据结构存放。

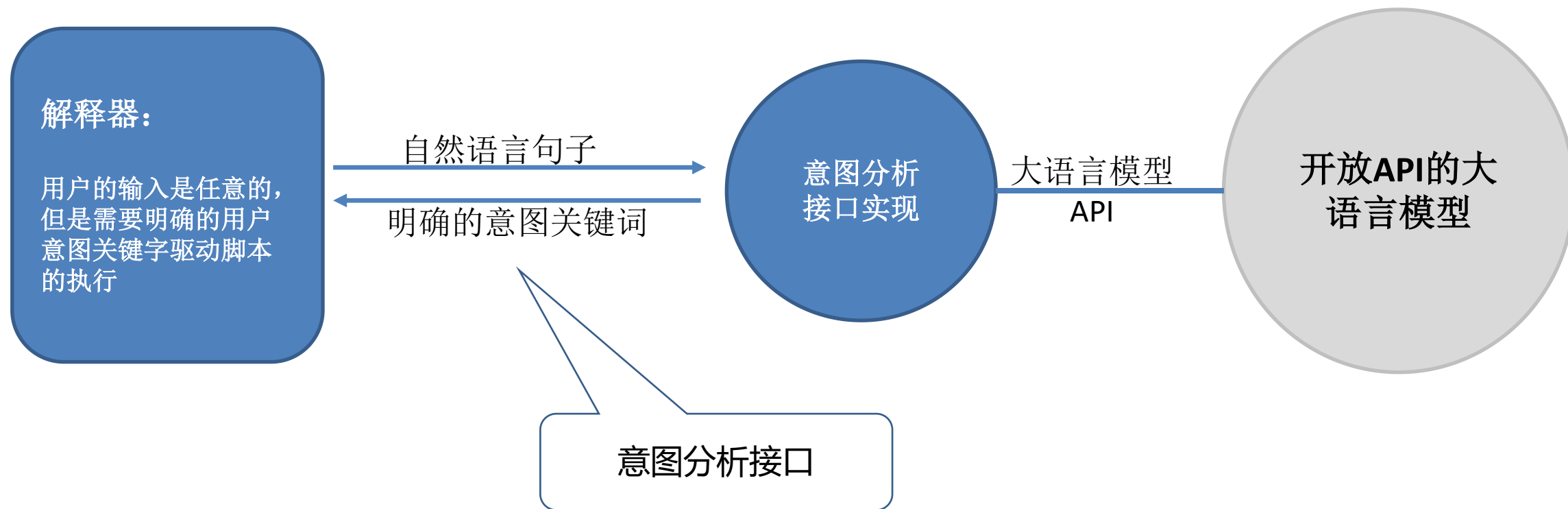


Interpreter的执行环境



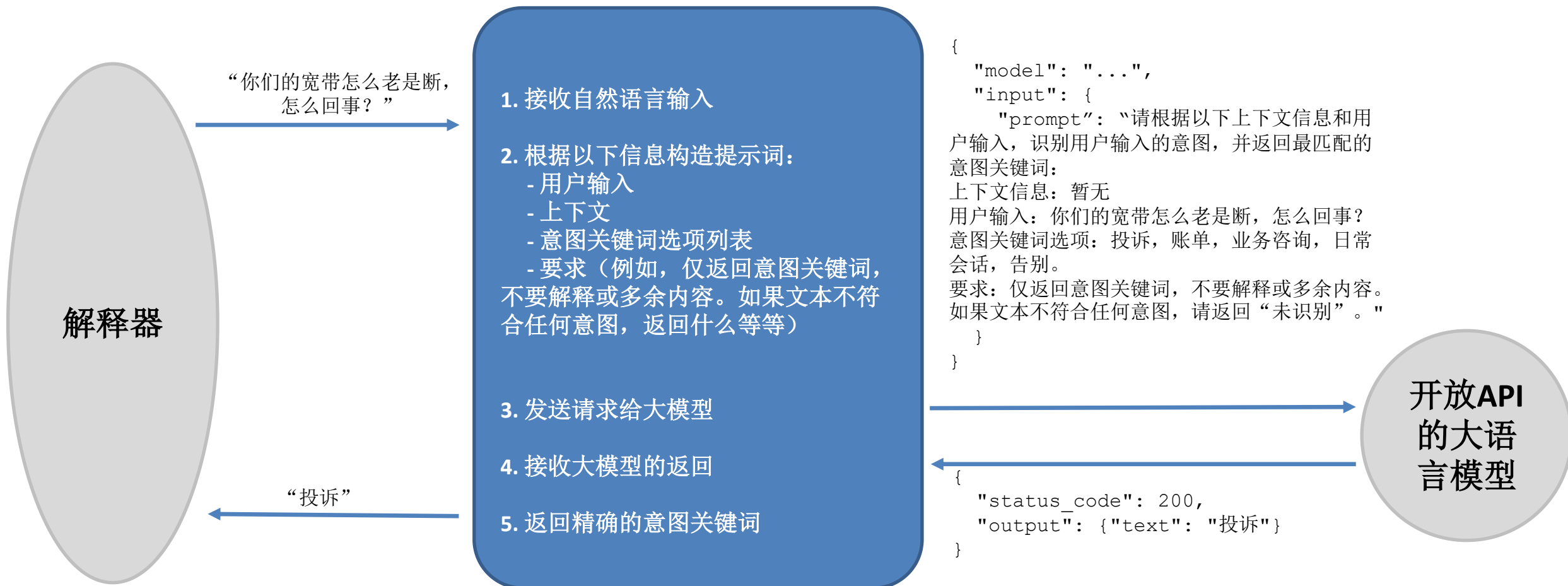


意图分析接口



意图分析接口

意图分析接口实现模块

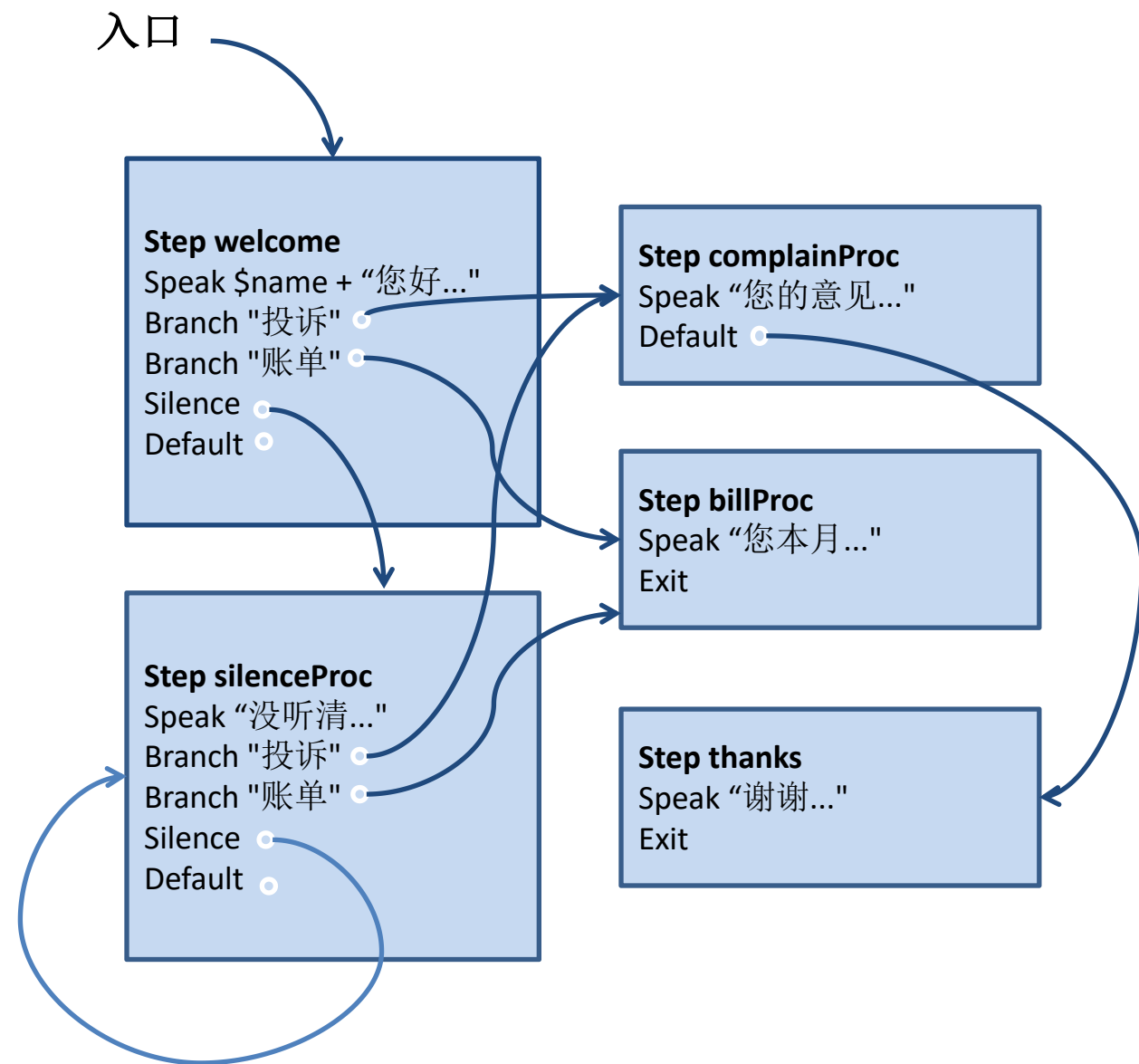




```
$ rsl test.rsl name=张先生 account=70
```

解释执行这个脚本 (test.rsl)

入口

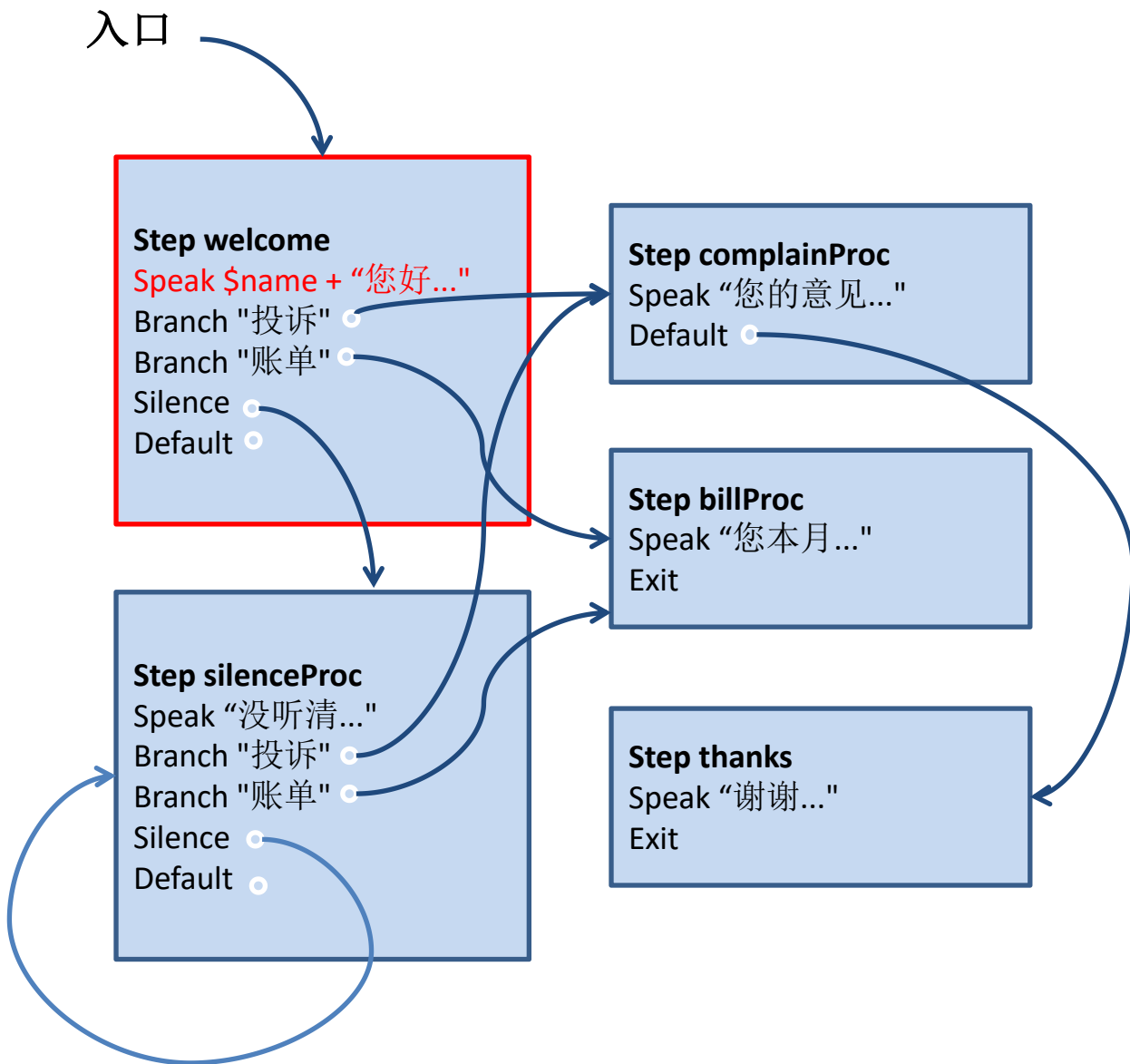


```
$ rsl test.rsl name=张先生 account=70
```

机器人：张先生您好，请问有什么可以帮您？

从入口welcome这个step开始执行，首先执行Speak，输出信息。

入口





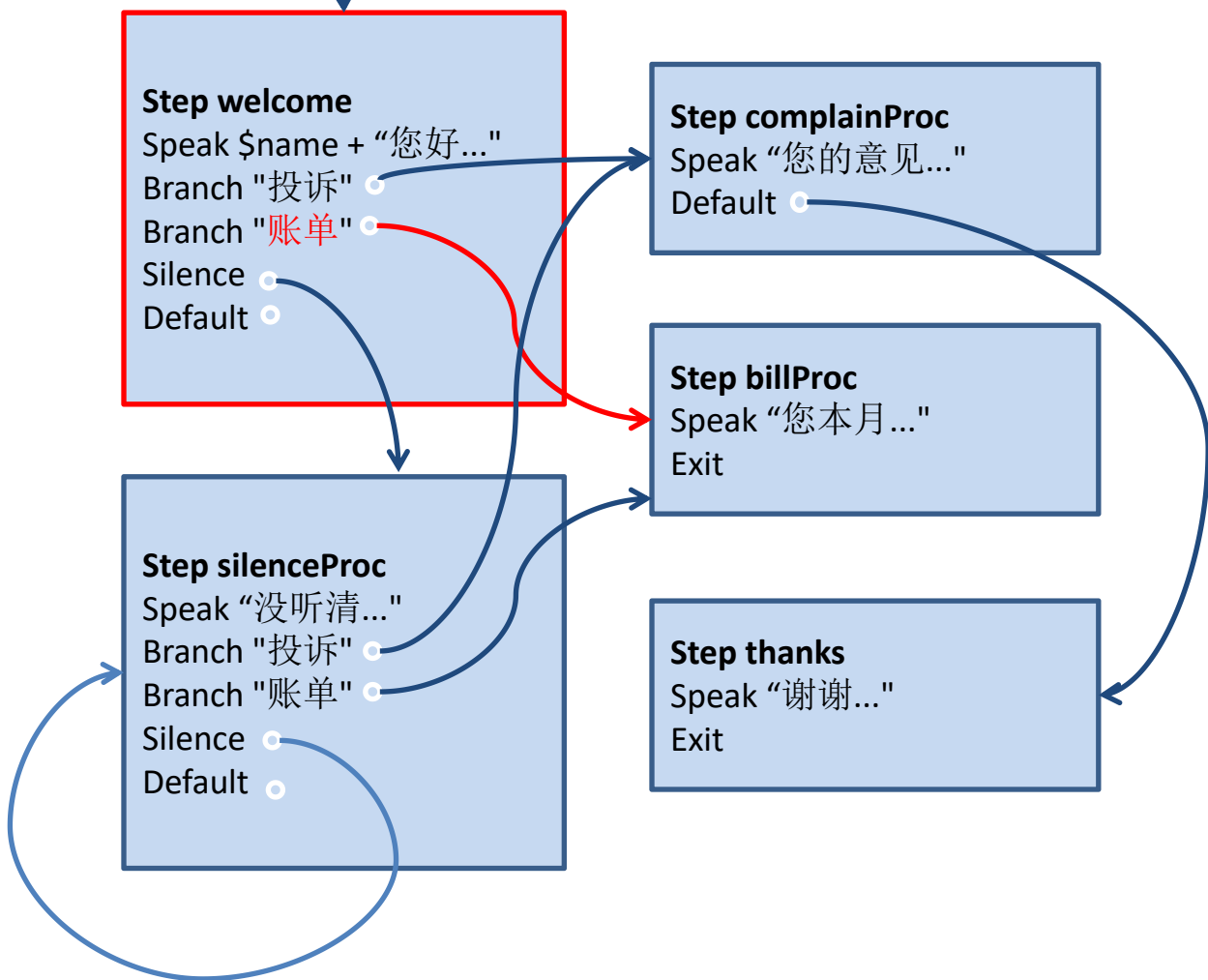
```
$ rsl test.rsl name=张先生 account=70
```

机器人：张先生您好，请问有什么可以帮您？

用户>>> 请问我这个月的费用是多少？

用户输入信息，根据用户输入调用大模型分析，得到精确意图为“账单”，根据Branch设置跳转到billProc

入口





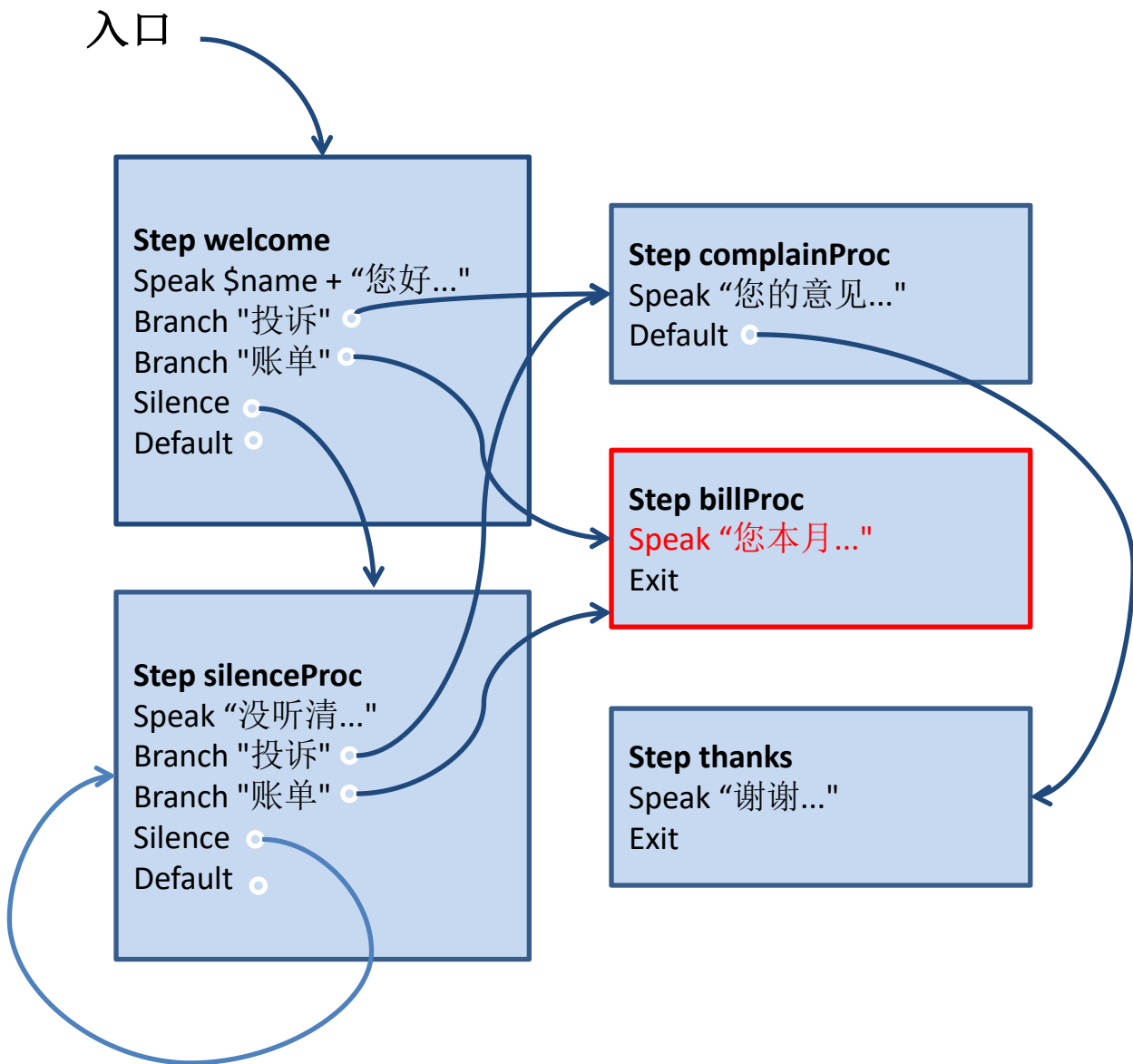
```
$ rsl test.rsl name=张先生 account=70
```

机器人：张先生您好，请问有什么可以帮您？

用户>>> 请问我这个月的费用是多少？

机器人：您本月的账单是70元。

执行billProc这个step，执行Speak动作输出信息





```
$ rsl test.rsl name=张先生 account=70
```

机器人：张先生您好，请问有什么可以帮您？

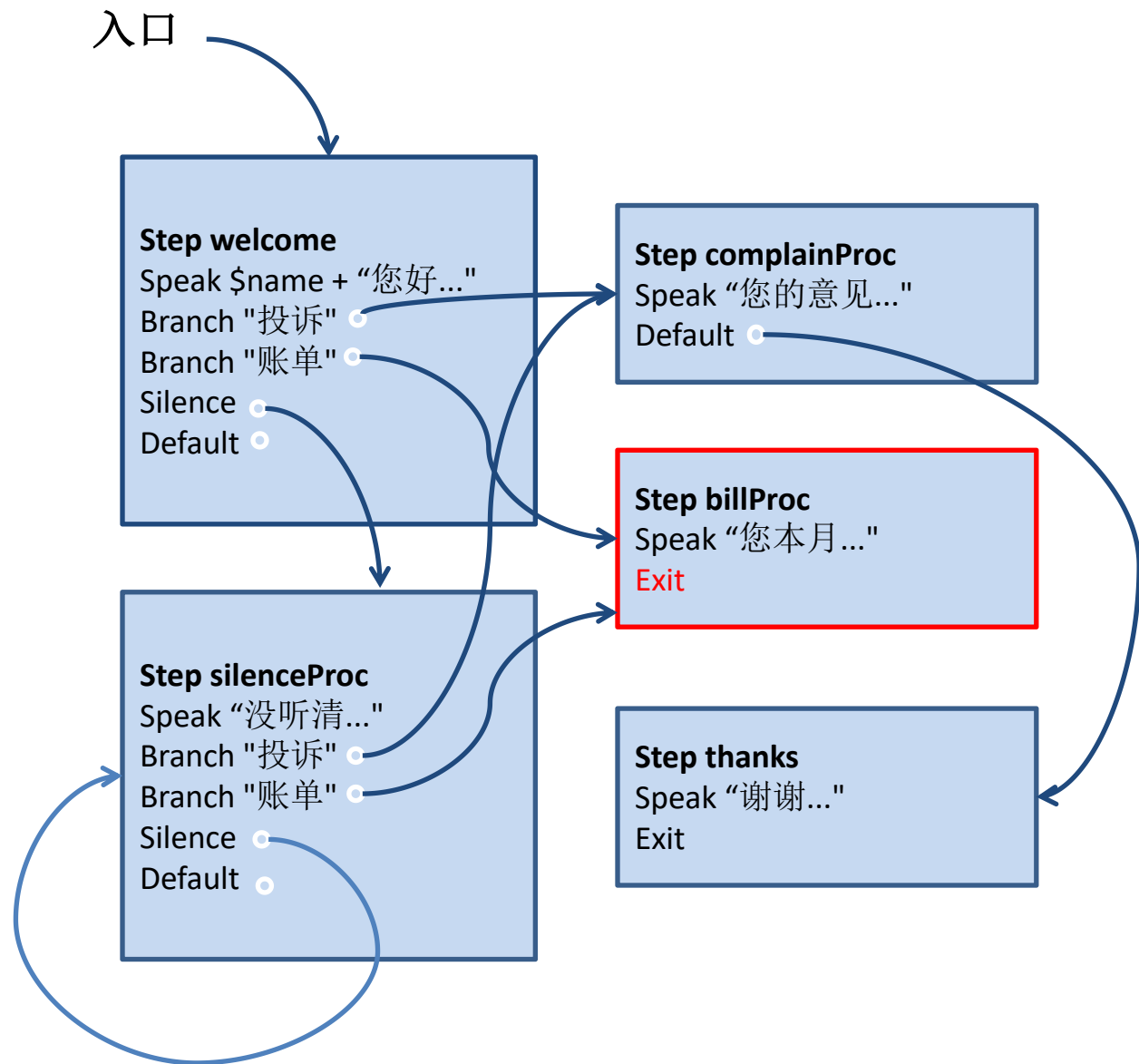
用户>>> 请问我这个月的费用是多少？

机器人：您本月的账单是70元。

\$

这个step并没有进一步的输入要求，执行Exit退出

入口





思考题

执行环境：

- 变量表 ?
- 当前Step

执行环境中的变量表应该是什么数据结构？如何构建？
如何使用？